Final Implementation Report

Intent-Aware Deception System for Real-Time Attacker Profiling · generated from docs/FINAL_REPORT.md

Final Implementation Report

Intent-Aware Deception System for Real-Time Attacker Profiling

Project type: BE (Information Science) Major Project - **Version:** 1.0 - **Status:** Complete and running end-to-end

> This report is written to be **honest and reproducible**. Every performance number is generated by scikit-learn on genuine held-out data and can be regenerated with `scripts/generate_eval_assets.py`. Every runtime claim can be reproduced against the live stack with `scripts/verify_live.py`.

1. Abstract

The Intent-Aware Deception System is a real-time defensive-security platform that captures attacker activity inside an isolated SSH honeypot, infers **why** the attacker is acting (their *intent*) rather than merely logging *what* they typed, and presents a live, explainable threat picture to a security analyst. Each captured command is passed through a hybrid classifier -- a deterministic rule engine, a calibrated machine-learning model, and an optional large-language-model semantic layer -- that assigns one of ten MITRE-aligned intent categories, a calibrated confidence, an attack stage, and an explainable risk score in the range 0-100. Sessions and source hosts are aggregated into evolving attacker profiles. The machine-learning model achieves **94.49 % accuracy** and **0.9229 macro-F1** on a leakage-controlled held-out test set of 272 commands, and a deliberately stricter leave-families-out evaluation is also reported (67.87 % accuracy) to characterise generalisation honestly. The entire system runs as a reproducible Docker stack.

2. Problem Statement and Objectives

Traditional honeypots and IDS tooling answer *"what commands were run?"* but leave the analyst to manually infer *"what is the attacker trying to achieve, and how dangerous is this session?"*. The objectives of this project are:

1. Capture attacker interaction safely, with **zero risk to the host**.
2. Classify each action into an **intent** taxonomy aligned with MITRE ATT&CK tactics.
3. Make **machine learning a core, honestly-measured** component -- not a bolt-on.
4. Produce an **explainable** risk score, so every alert can be justified to an analyst.
5. Aggregate activity into **attacker and session profiles** that evolve in real time.
6. Deliver a **live analyst dashboard** over REST + WebSocket.
7. Be fully **reproducible** (Docker, pinned dependencies, fixed random seed).

3. Background and Motivation

A honeypot is a decoy service with no legitimate users, so *all* traffic to it is, by definition, suspicious. This project uses **Cowrie**, a medium-interaction SSH honeypot that **emulates** a shell: it returns believable output but never executes anything, making it a safe source of rich attacker telemetry. MITRE ATT&CK provides the industry-standard vocabulary of adversary *tactics* (Reconnaissance, Discovery, Credential Access, ...); mapping intent onto these tactics makes the system's output immediately meaningful to practitioners and lets a sequence of intents be read as an attacker's progression through the kill chain.

4. System Architecture and Data Flow

The system is a streaming pipeline. Each stage is a small, independently-tested service:

```
Attacker -> Cowrie honeypot (isolated) -> cowrie.json (append-only log) -> async Log Collector (tails the file, survives rotation) -> Event Normalizer (Cowrie schema -> internal NormalizedEvent) -> Feature Extractor (session-level behavioural features) -> Hybrid Intent Classifier ■■ Rule engine ■■ ML model (calibrated) ■■ Gemini semantic layer (optional) -> Attack-Stage mapping (MITRE tactic) -> Risk Engine (explainable 0-100 score + severity) -> Session Aggregator + Attacker Profiler -> MongoDB (persistence) -> FastAPI ■■ REST endpoints ■■ WebSocket broadcast -> React dashboard (live)
```

Relevant code: `backend/app/pipeline.py`, `backend/app/services/`.

5. Intent Taxonomy and MITRE ATT&CK Mapping

Ten intent classes are defined in `backend/app/models/intent.py`:

#	Intent	MITRE tactic	Example command
1	RECONNAISSANCE	Reconnaissance (TA0043)	`uname -a`, `whoami`
2	DISCOVERY	Discovery (TA0007)	`ps aux`, `netstat -antp`
3	CREDENTIAL_ACCESS	Credential Access (TA0006)	`cat /etc/shadow`
4	EXECUTION	Execution (TA0002)	`wget http://.../payload.sh -O /tmp/p.sh`
5	PRIVILEGE_ESCALATION	Privilege Escalation (TA0004)	`sudo su -`
6	PERSISTENCE	Persistence (TA0003)	`echo ... >> ~/.ssh/authorized_keys`
7	EXPLOITATION	Execution (TA0002)	`() { :;}; echo VULNERABLE`
8	EXFILTRATION	Exfiltration (TA0010)	`tar czf /tmp/loot.tar.gz /etc /home`
9	CLEANUP	Defense Evasion (TA0005)	`rm -rf /var/log/auth.log`
10	OTHER	Initial Access / Unknown	session/login telemetry, benign input

Every prediction the system emits carries an **intent, confidence, source, and evidence** -- this is a hard invariant enforced through the models.

6. Machine Learning Design and Honest Evaluation

This is the core of the system. Design (`backend/app/ml/train.py`, `backend/app/ml/intent_ml_classifier.py`):

- **Features:** a `FeatureUnion` of TF-IDF word n-grams (1-2) and character n-grams (`char_wb`, 3-6). Character n-grams are essential for shell commands, where flags, paths and obfuscation matter at the sub-token level.
- **Model:** `CalibratedClassifierCV(LinearSVC)` -- a linear SVM (strong on sparse high-dimensional text) wrapped in probability calibration so the confidences it reports are meaningful, not raw margins. Registered internally as `linsvc_calibrated`.
- **Selection:** three candidates (`linsvc_calibrated`, `logreg`, `complement_nb`) compared by 4-fold stratified cross-validation on the training split (`f1_macro`) **before** the test set was touched.

Honest results (full detail and figures in `docs/evaluation/evaluation_report.md`):

Evaluation	Accuracy	Macro-F1	Notes
Primary held-out (272 test)	**94.49 %**	**0.9229**	no identical command shared across split
Robustness (leave-families-out)	67.87 %	0.5267	stricter by design; novel templates

- **No leakage:** the training script *asserts* that no identical command string appears in both train and test; the robustness split holds out whole template families.
- **Not hidden:** the weakest class (RECONNAISSANCE, F1 0.714) and the lower robustness number are reported, not suppressed.
- **Reproducible:** `RANDOM_SEED = 1337`; re-running `train.py` reproduces the numbers.

The trained model is persisted as a joblib bundle and loaded lazily at runtime. If the artifact is missing, the system degrades gracefully to rules + Gemini rather than failing.

7. Hybrid Classification and Arbitration

The hybrid classifier (`backend/app/services/hybrid_classifier.py`) combines three sources with an explicit precedence, and records which source decided each event (`RULE` / `ML` / `HYBRID` / `GEMINI` / `FALLBACK`):

1. **Strong rule signature** (a rule matches with confidence >= 0.90 and is not OTHER) is authoritative -- deterministic, auditable, instant.

2. Otherwise a **confident ML prediction** (calibrated confidence >= 0.40) is used; if the rule agrees, the source is recorded as `HYBRID`.
3. **Gemini is consulted only when the rule is weak *and* ML is not confident** -- i.e. for genuinely novel/ambiguous input. It is never on the hot path for known attacks.
4. Best-available fallback, then `OTHER`, guarantee the pipeline always produces a labelled, explainable result.

Gemini is strictly optional: the system starts and runs fully without an API key. In the live end-to-end demonstration, ML/HYBRID classified 21/21 commands and Gemini was never invoked -- demonstrating the design goal that the LLM is an enhancement, not a dependency.

8. Explainable Risk Scoring

The risk engine (`backend/app/services/risk_engine.py`) produces a 0-100 score as an **intent base weight** plus **behavioural escalations** (e.g. >= 3 failed logins, >= 5 commands in a session, >= 4 distinct event types), clamped to 0-100. Each contribution is retained as human-readable evidence, so an analyst can see *why* a score was assigned. Scores map to severity tiers:

Score	Severity
0-24	LOW
25-49	MEDIUM
50-74	HIGH
>= 75	CRITICAL (raises a threat alert)

9. Attacker and Session Profiling

Events are aggregated into **sessions** (a single SSH connection: dominant intent, current attack stage, command count, evolving risk) and **attacker profiles** keyed by source host (sessions seen, intent history, peak risk). This turns a flat event log into a narrative: *this host connected, ran discovery, escalated privileges, then attempted exfiltration.* Code: `backend/app/services/session_aggregator.py`, `backend/app/services/attacker_profiler.py`.

10. Backend Implementation

FastAPI application with eight routers (`backend/app/api/routers/`): `health`, `dashboard`, `events`, `sessions`, `attackers`, `analytics`, `alerts`, and a `websocket` broadcaster. Persistence is synchronous **pymongo** into MongoDB; models are **pydantic v2**. The `/api/health` endpoint reports live component status (DB, ML model + accuracy, rule engine, Cowrie feed, Gemini availability) so the dashboard and the verifier can confirm the system's real state at any moment.

11. Frontend Dashboard

React 19 + Vite + Tailwind + recharts, in a dark SOC theme (`frontend/src/`). It provides a live overview (stats, recent events, intent/risk charts), session and attacker inspection pages, an analytics view, and a threat alerts page. The navbar shows real-time status badges for ML, AI (Gemini), the honeypot feed and the database, driven by `/api/health`; recent-event rows show the **classification source** and ML confidence, making the hybrid decision visible to the analyst. A WebSocket connection streams new events into the UI without polling.

12. Honeypot Integration and Security Isolation

Security is a first-class constraint, not an afterthought:

- Cowrie is the **only** component that ingests attacker commands, and it **emulates** the shell -- nothing an attacker types is ever executed on the Windows host or the backend.
- The backend mounts the Cowrie log **read-only** and treats every field as untrusted data.
- Captured commands are sanitised before storage/display (dedicated tests in `tests/test_sanitization.py`).
- Secrets live only in `.env` (never committed); `.env.example` documents the shape.
- The demonstration uses **RFC-5737** documentation IP ranges (192.0.2.0/24, 198.51.100.0/24) which route nowhere real, and no real malicious payloads are executed.

13. Deployment and Reproducibility

A single `docker compose up -d` builds and starts four services -- `mongo`, `cowrie` (SSH on 127.0.0.1:2222), `api` (127.0.0.1:8000), and `dashboard` (127.0.0.1:5173) -- all bound to localhost. Host and container pin identical scientific-stack versions (scikit-learn 1.9.0, numpy 2.5.2, scipy 1.18.1, joblib 1.5.3) so the serialised model loads identically in both. Configuration is via `.env`; the pipeline auto-starts inside the API

container.

14. Testing and Live Verification

- **Automated tests:** 74 tests across 17 modules (`tests/`) -- normalization, feature extraction, rule/ML/Gemini/hybrid classification, risk, persistence, aggregation, WebSocket streaming, sanitisation, and an end-to-end pipeline test. **All 74 pass.**
- **Live end-to-end proof:** `scripts/demo_all_intents.py` drives 18 commands spanning all 10 intents through a real SSH session into Cowrie; `scripts/verify_live.py` then queries the API and confirms the pipeline is live. Latest run: **29 events persisted, all 10 intent classes observed, 21/21 commands classified by the ML model (ML or HYBRID), one session escalated to CRITICAL** -- verdict **PASS (8/8 checks)**.

15. Limitations, Ethics and Responsible Use

- The ML model generalises less well to command *families* it has never seen (67.87 % leave-families-out); this is expected for a lexical model and is exactly why the hybrid rule + LLM layers exist. Reported transparently.
- Risk weights are project-defined and reasoned, not empirically calibrated against a labelled incident corpus.
- The dataset, while de-duplicated and family-aware, is synthetic/curated rather than a large captured corpus; expanding it with real (sanitised) honeypot captures is future work.
- **Ethics:** this is a **defensive** system operated on infrastructure the defender owns. It must only be deployed on authorised systems; the honeypot is isolated and captured commands are never executed.

16. Conclusion and Future Work

The system meets every stated objective: it captures attacker activity safely, classifies intent with a genuinely-measured ML core (94.49 % held-out), explains its risk scores, profiles attackers in real time, and presents everything on a live dashboard -- all reproducibly, with honest metrics and end-to-end proof. **Future work:** enlarge and diversify the training corpus with real sanitised captures; add a sequence model over the *order* of intents to predict the attacker's next move; calibrate risk weights against labelled incidents; and add automated deception responses (adaptive fake filesystem content) driven by the inferred intent.

Reproduce the metrics: `python scripts/generate_eval_assets.py` - *Reproduce the live proof:* `docker compose up -d && python scripts/demo_all_intents.py && python scripts/verify_live.py`